SPECTEROPS
Creators of BloodHound

From Dev to Prod

Walking a cross-account attack path

Julian Catrambone, Daniel Heinsen
SpecterOps

Julian Catrambone – @n0pe_sled

Senior Offensive Security Engineer

- Red team operator and TRACER (Tradecraft Research and Capability Engineer)
- Experience consulting with Fortune 100 clients red teaming, penetration testing and detection engineering
- Previous internal corporate red team experience
- Co-Creator of AWS in the Bloodhound platform

Daniel Heinsen – @hotnops

Principal Security Researcher

- Security Researcher / Software Developer
- Entra ID / Hybrid
- A little bit of AWS
- I really like learning new things
- Hiking / Being Outside / Dad Stuff
- Co-Creator of AWS in the Bloodhound platform

Agenda

01 AWS Primer

02 AWSHound and OpenGraph

03 Capstone exercise

04 Reconstructing the Attack Path

05 Tradecraft Academy

AWS Primer

Explaining AWS permissions evaluation

Every AWS API Request has Five Deciding Inputs

When a call fails, name the input that differs from what the policy expects

Principal

What is attempting to execute the action.

Action

The exact API action, with its namespace.

Resource

The ARN the action is evaluated against.

Policy layers

Identity, boundary, session, SCP, trust, resource.

Runtime context

Tags, external ID, service path, encryption context

Principal: The Identity Taking Action

Name the exact session, not the human behind the keyboard

Who is accessing the API

- IAM User
- IAM Role
- Assume Role Session
- AWS Account Principal
- AWS Service
  - ec2
  - lambda
  - cloudformation
  - eks

Examples of principals

```
arn:aws:iam::111122223333:user/build-svc

arn:aws:iam::111122223333:role/OrganizationAccountAccessRole

arn:aws:sts::111122223333:assumed-role/DeployRole/i-0abc123

arn:aws:iam::111122223333:root

lambda.amazonaws.com

ec2.amazonaws.com
```

Action: the exact API operation

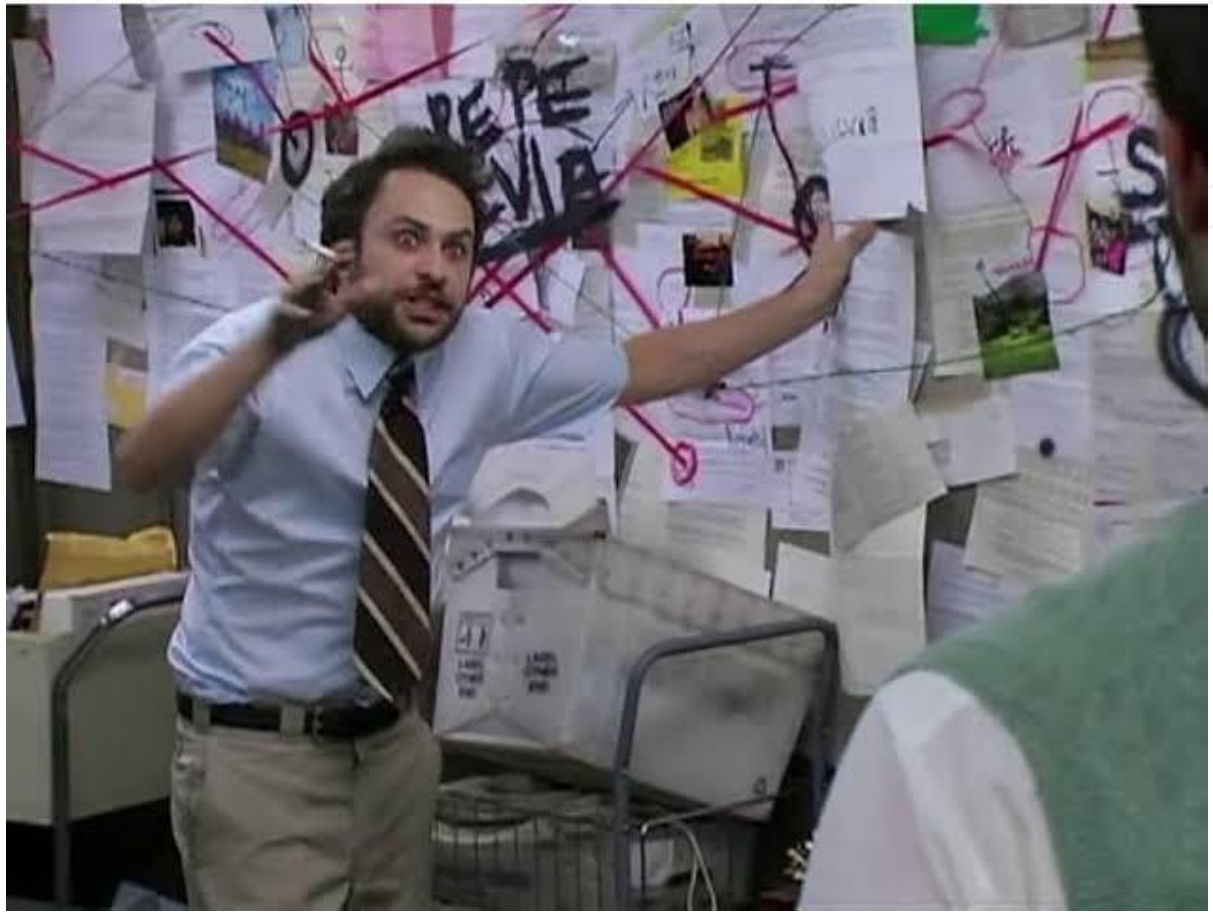
Policies match action strings, not what you meant to do

What is an action?

- There are over **470** services and **21,000** actions in AWS
  - This number is growing everyday

- Actions are always formatted: ***namespace:Operation***

- Wildcards expand these actions dramatically
  - *:* expands to all 21,000 actions
  - iam:* expands to all actions in the IAM namespace or service



Resource: the ARN under evaluation

Every action is checked against one specific ARN

What is a Resource?

- Resource is the object being acted on
  - S3 Buckets, EC2 Instances, etc.
- Typically specified by ARN though these ARN's can take different forms
  - S3 ARN's do not include account id's
  - Wildcards expand depending on location
- Resources can be **cross-account**!

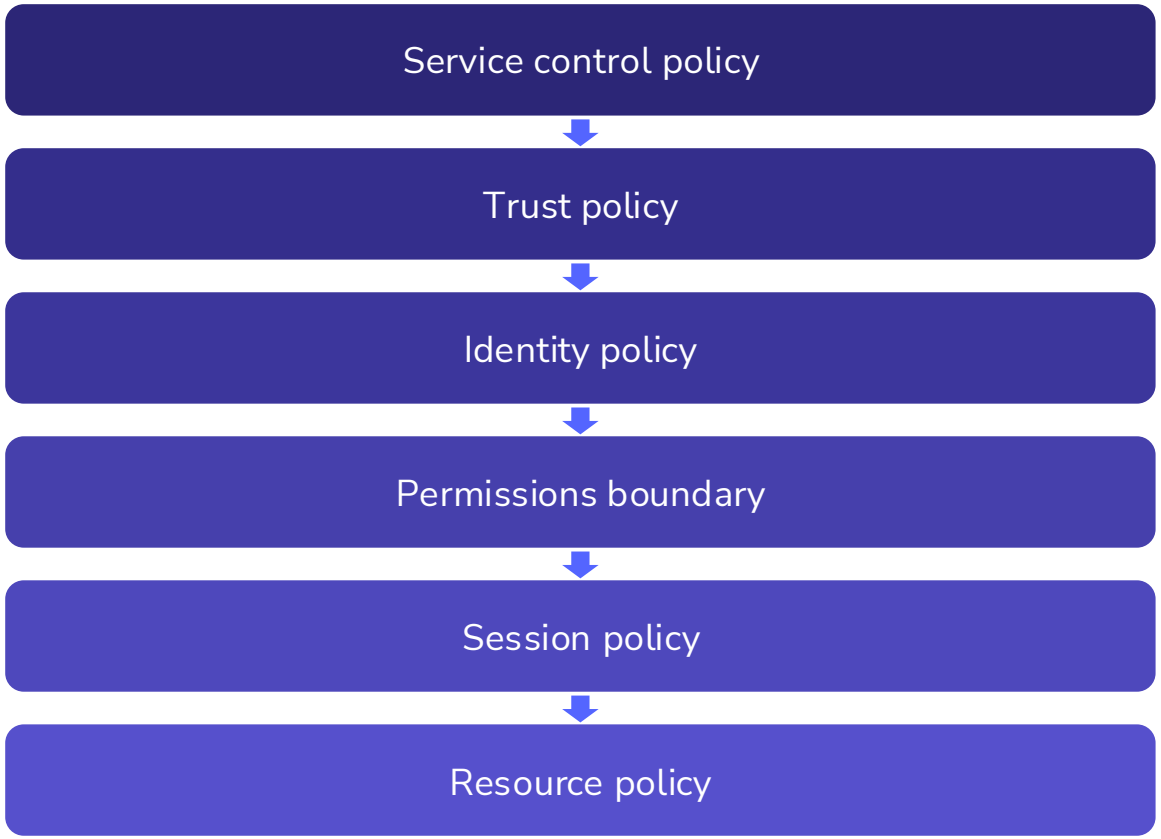
Examples of resource ARNs

```
arn:aws:s3:::evidence-bucket

arn:aws:s3:::evidence-bucket/*

arn:aws:iam::111122223333:role/dev-lambda-role

arn:aws:lambda:us-east-1:111122223333:function:report-builder

arn:aws:kms:us-east-1:111122223333:key/<key-id>

*
```

Policy layers: every document that applies

One request can be evaluated by five different policies

Evaluated top to bottom



Runtime context

The same principal and action can pass or fail on context alone

What is the Runtime Context?

- Context of the Identity taking the action

Common Context Conditions:

- External-ID
- ViaService
- Source IP
- Tags
- MFA

```
"Effect": "Allow", "Action": "sts:AssumeRole",
"Resource":
"arn:aws:iam::111122223333:role/DeployRole",
"Condition": {
   "StringEquals": {"sts:ExternalId": "7f3c-
9a21"},
   "Bool": {"aws:MultiFactorAuthPresent":
"true"} }
```


Explicit Deny Overrides Every Allow

A visible action in one policy may still be unusable once every applicable layer is evaluated

Every request starts at implicit deny. A relevant Allow authorizes it. Any applicable explicit Deny ends it, no matter which layer carries the Deny.

Identity policies dictate permissions

Attached to the principal, additive across every policy, and easy to over-scope

How an identity policy grants permission

- Attached to an IAM user, group, or role
- Every statement pairs Action with Resource, plus an optional Condition.
- Allow is additive across all attached policies
- Explicit denies at any stage of evaluation result in a deny

Example Identity Policy

```
"Version": "2012-10-17",
"Statement": [{
  "Effect": "Allow",
  "Action": "lambda:UpdateFunctionCode",
  "Resource": "arn:aws:lambda:*:*:function:report-*"
}]
```


Permissions boundaries cap what a policy can grant

A boundary never grants access; it sets the ceiling an identity policy can reach

How a permissions boundaries work

- Dictate the ***maximum*** permissions a principal can have
- Effective access is the intersection of the identity policy and the boundary.
- An action the boundary omits is denied even where a policy allows it.
- Boundaries **do not** constrain resource policies or service-linked roles.

Example Permissions Boundary

```
"Version": "2012-10-17",
"Statement": [{
  "Effect": "Allow",
  "Action": "lambda:*",
  "Resource": "arn:aws:lambda:*:*:function:report-*"
}]
```


Service control policies subtract across the whole account

An organization policy can only remove access, and no local admin can undo it.

How an SCP works

- SCPs are used to limit the effective permissions of all principals in an account
- Applied at the organization, OU, or account level
- A Deny anywhere in the chain ends the decision, whatever the identity policy says.
- SCPs do not apply to the management account or to service-linked roles.

Example: a tag-conditioned deny

```
"Effect": "Deny",
"Action": "lambda:UpdateFunctionCode",
"Resource": "*",
"Condition": {
  "StringEquals": {
    "aws:ResourceTag/env": "prod"
  }
}
```


Resource-side controls answer from the other end

The resource decides too, and cross-account access needs both sides to agree

How resource policies work

- Attached to the resource, not the caller:
  - buckets, keys, functions, queues, topics.
- Evaluation rules are different depending on context
  - Same account
    - either the identity policy or the resource policy can allow.
  - External account
    - Both the identity policy and the resource policy must allow

Example: a function resource policy

```
"Effect": "Allow",
"Principal": {
  "AWS": "arn:aws:iam::444455556666:root"
},
"Action": "lambda:InvokeFunction",
"Resource": "arn:aws:lambda:...:report-builder"
```


AWS Primer

How permissions become attack paths

AWS Attack Paths are Identity Chains

Always know who is acting now and which identity becomes reachable next

Principal
who is acting

→

API action
what is called

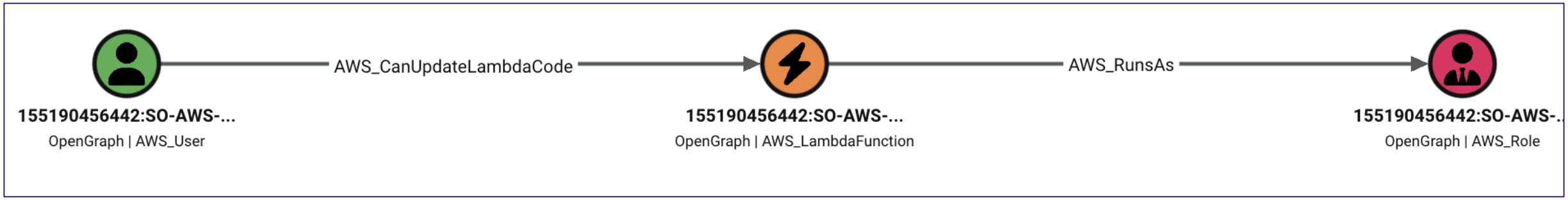
→

Resource or service
what runs the code

→

New identity
what you become

A principal calls an AWS API. A policy permits or blocks the call.
A service may run as another role. Temporary credentials expose the next identity.



Change the workload, reach its identity

Lambda

- The function runs as its execution role.
- lambda:UpdateFunctionCode replaces the code that role runs.

EC2

- The instance carries an instance profile.
- Code execution on the host reaches IMDS and the role credentials.

EKS

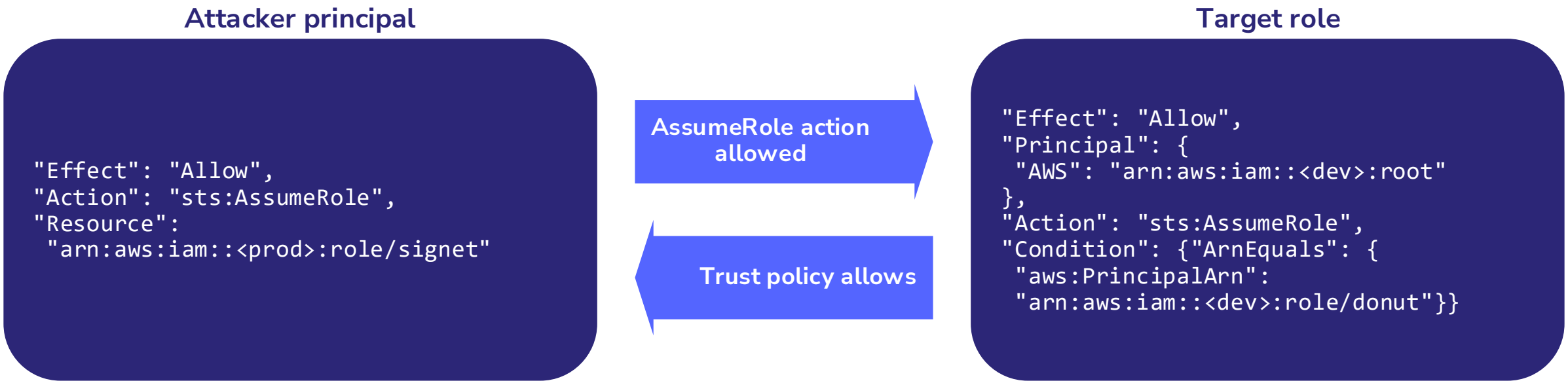
- A Pod Identity association binds a role to a service account.
- A pod on that service account receives projected credentials.

Cross-account AssumeRole

Your identity policy allows the call; the target's trust policy decides who gets in

The two-door model

- Source Principal: an identity policy allowing sts:AssumeRole on the target ARN.
- Target Principal: the trust policy, a resource policy that lives on the role.



AWSHound and OpenGraph

Generating Attack Paths

SPECTEROPS

AWSHound Graphs Cross Account Attack Paths in AWS

What it actually does

- Collects raw configuration data across critical AWS services
- Evaluates cross account identity policies for every principal and resource in your AWS organization
- AWSHound Evaluates:
  - Identity Policies
  - Conditional Access Controls
  - Resource Policies
  - Service Control Polices
  - Trust Policies
  - Permissions Boundries

```
goawshound — collect AWS data and build a BloodHound OpenGraph.

usage:
  goawshound <command> [flags]

commands:
  collect   fetch AWS data into a raw JSON directory (AWS -> raw/)
  process   build the graph from raw collect JSON (raw/ -> built.json)
  emit      encode the built graph as OpenGraph JSON (built.json -> graph.json)
  all       run every phase, persisting each artifact (AWS | raw/ | GAAD -> graph.json)

examples:
  goawshound all     --profile prod -o graph.json              # collect one profile + build + emit
  goawshound all     --org-role OrganizationAccountAccessRole  # collect the whole AWS Organization
  goawshound all     --aws-config ~/.aws/config -o graph.json  # collect + build + emit
  goawshound collect --aws-config ~/.aws/config -o raw/        # AWS -> raw JSON only
  goawshound all     --input-dir raw/ -o graph.json            # build from collected JSON
  goawshound process --input-dir raw/ -o built.json            # build only
  goawshound emit    -i built.json -o graph.json               # encode only

Live collection chooses accounts from --profile (repeatable), the --aws-config file (every profile whose section sets
"bloodhound_collect = true"), or --org-role <role> (assume that role across the whole AWS Organization, e.g.
OrganizationAccountAccessRole).

With no command, flags are treated as "all". Run "goawshound <command> -h" for a command's full flag list.
```


Traversable edges by service

36 traversable edges, weighted toward identity

IAM: 15

RunsAs
CanUpdateSelfPolicy
CanUpdateOtherPolicy
CanUpdateAssumeRolePolicyAndAssume
CanCreateAndAssumeAdminRole
CanCreateAndPassAdminRole
CanCreateAndAccessAdminUser
CanCreateAccessKey
CanCreateLoginProfile
CanUpdateLoginProfile
CanCreateOIDCProviderAndAssumeRole
CanCreateSAMLProviderAndAssumeRole
CanUpdateSAMLProvider
CanUpdateOIDCProviderThumbprint
CanPassRoleToService

STS: 3

CanAssumeRole
CanAssumeRoleWithSAML
CanAssumeRoleWithWebIdentity

Lambda: 4

CanUpdateLambdaCode
CanUpdateLambdaConfiguration
LambdaFunctionWrite
LambdaFunctionAll

EC2: 2

CanModifyEC2InstanceAttribute
EC2CanVolumeSwap

CloudFormation: 2

CanUpdateCloudFormationStack
CanExecuteCloudFormationChangeSet

EKS: 6

ClusterHasNodeGroup
NodeGroupHasRole
CanAssumeRoleViaIRSA
CanAssumeRoleViaPodIdentity
CanCreateAndAssociateEKSAccessEntry
CanCreateEKSPodIdentityAssociation

SSM: 2

SSMCanSendCommand
SSMCanStartSession

Organizations: 1

Contains

AWSHound Collection

- **collect**
  - Fetches IAM/authorization data into a directory of raw JSON
- **process**
  - Builds the graph, nodes keyed by raw ARN ids
- **emit**
  - Encodes the built graph as BloodHound OpenGraph JSON
- **all** — every phase in one invocation
  - Accounts from --profile, --aws-config, or --org-role

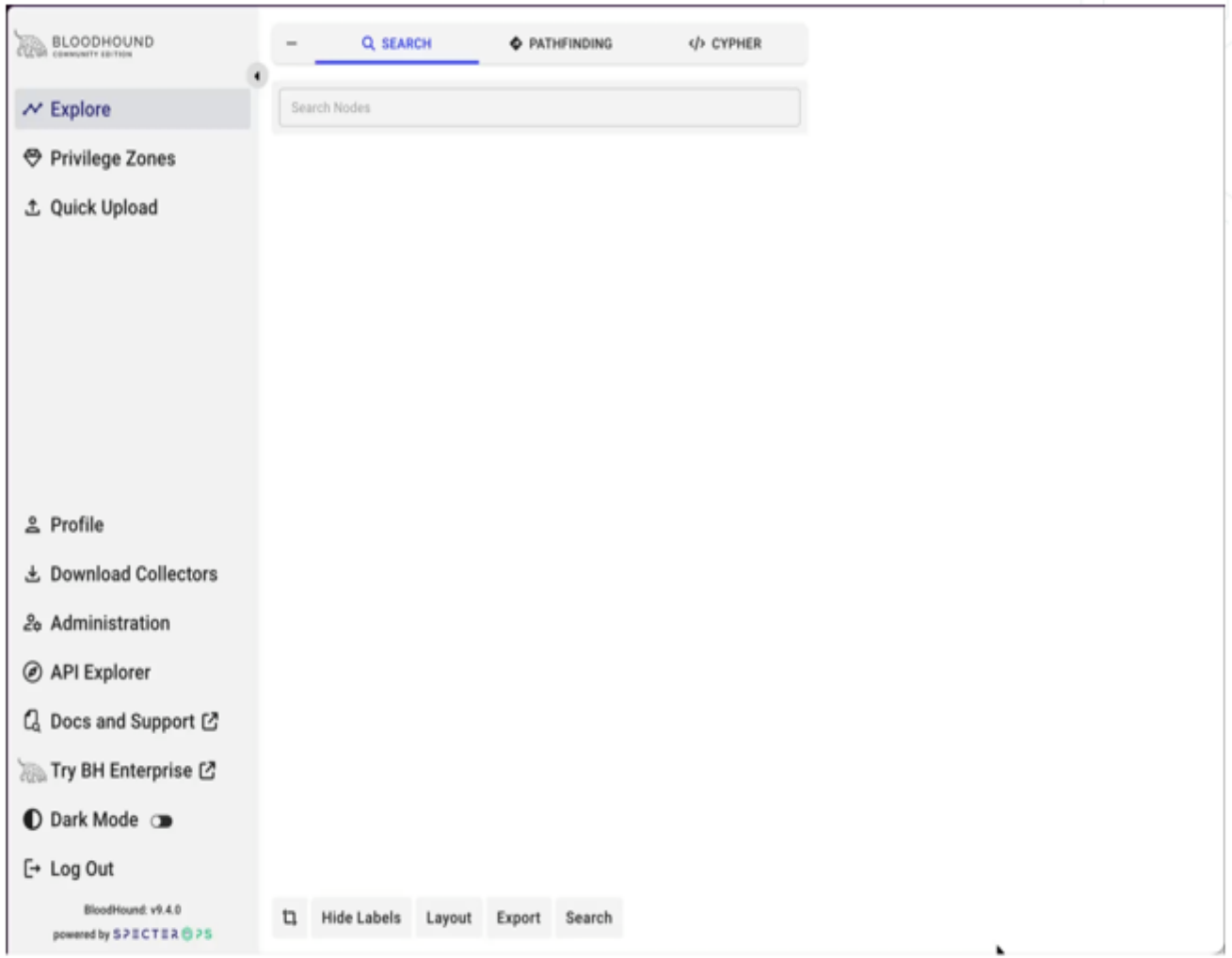
SPECTER PS

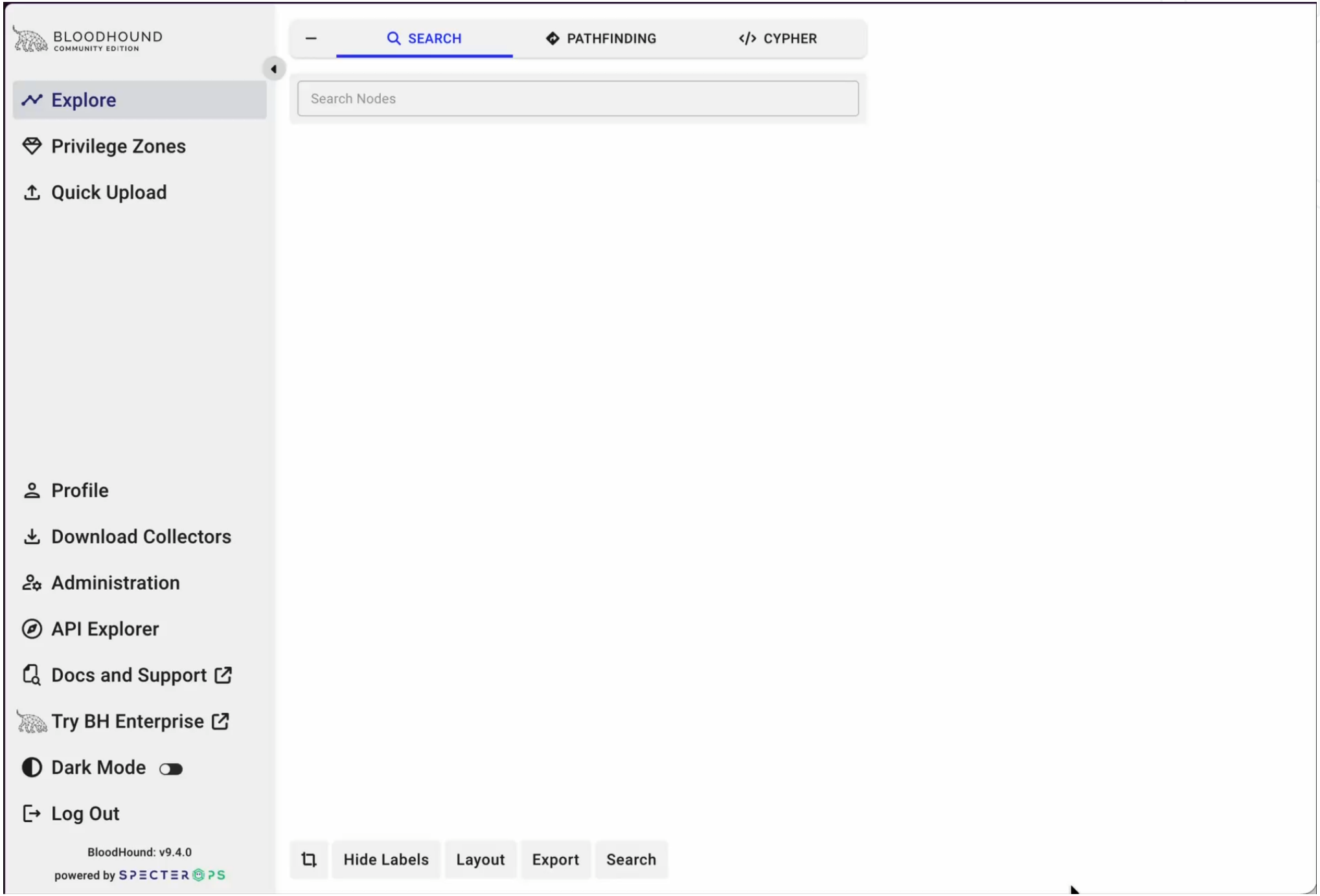
```
n0pe-sled@SpecterTop:~/Developer/Workshop/awshound-output:() > goawshound --help
```


Import the schema, then the graph

BloodHound needs the AWS kinds before the nodes mean anything

- **Enable OpenGraph**
  - Turn on the feature flag in BloodHound settings
- **Upload the schema**
  - 20 node kinds, 156 edge kinds, 41 traversable — with display names and icons
- **Upload graph.json**
  - File ingest in the UI, or push it with a BloodHound API token
- **Re-ingest freely**
  - The schema only registers once; the AWS namespace keeps kinds apart from AD and Azure

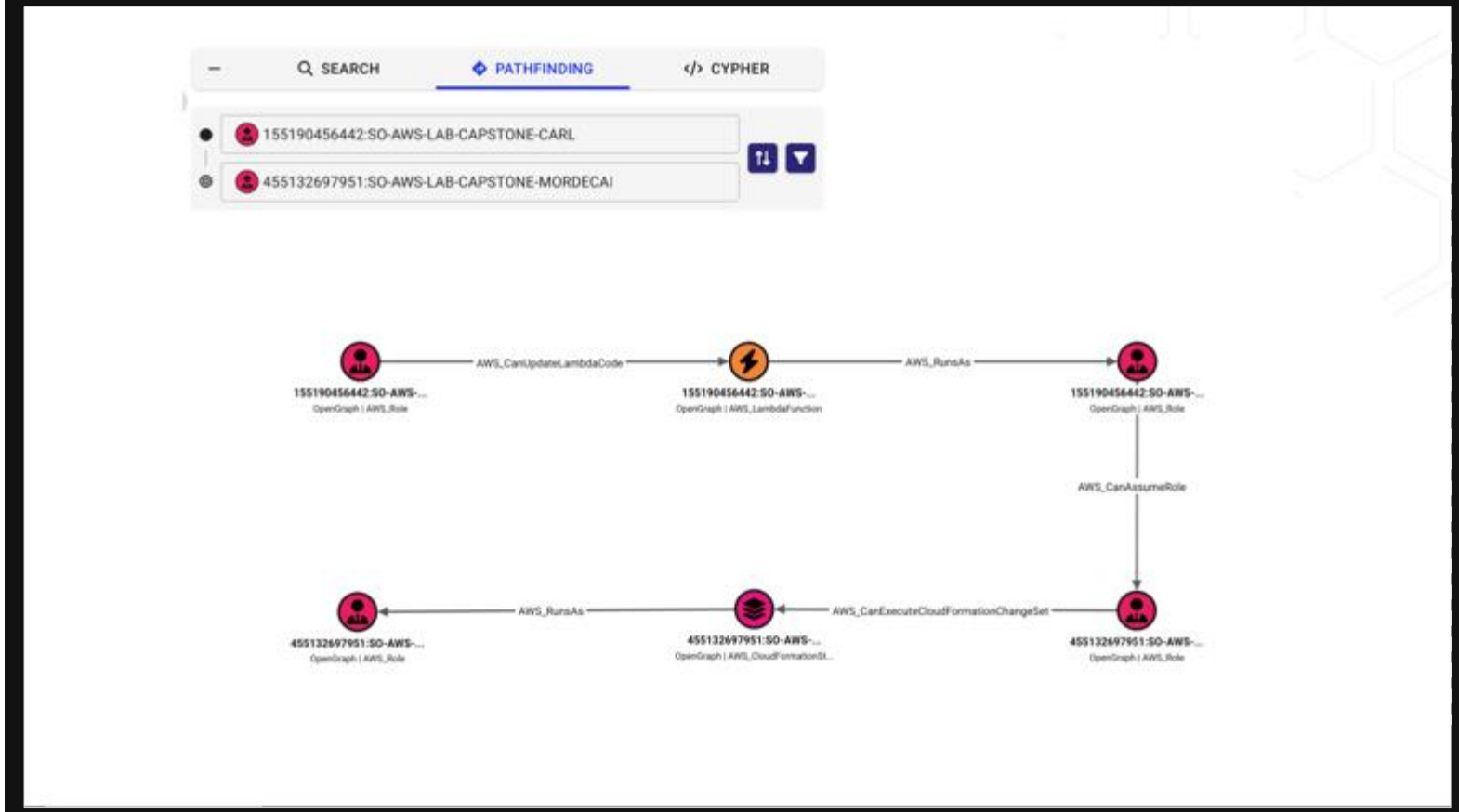



BLOODHOUND
COMMUNITY EDITION
SEARCH
PATHFINDING
CYPHER
Search Nodes
Explore
Privilege Zones
Quick Upload
Profile
Download Collectors
Administration
API Explorer
Docs and Support
Try BH Enterprise
Dark Mode
Log Out
BloodHound: v9.4.0
powered by SPECTER OPS
Hide Labels
Layout
Export
Search

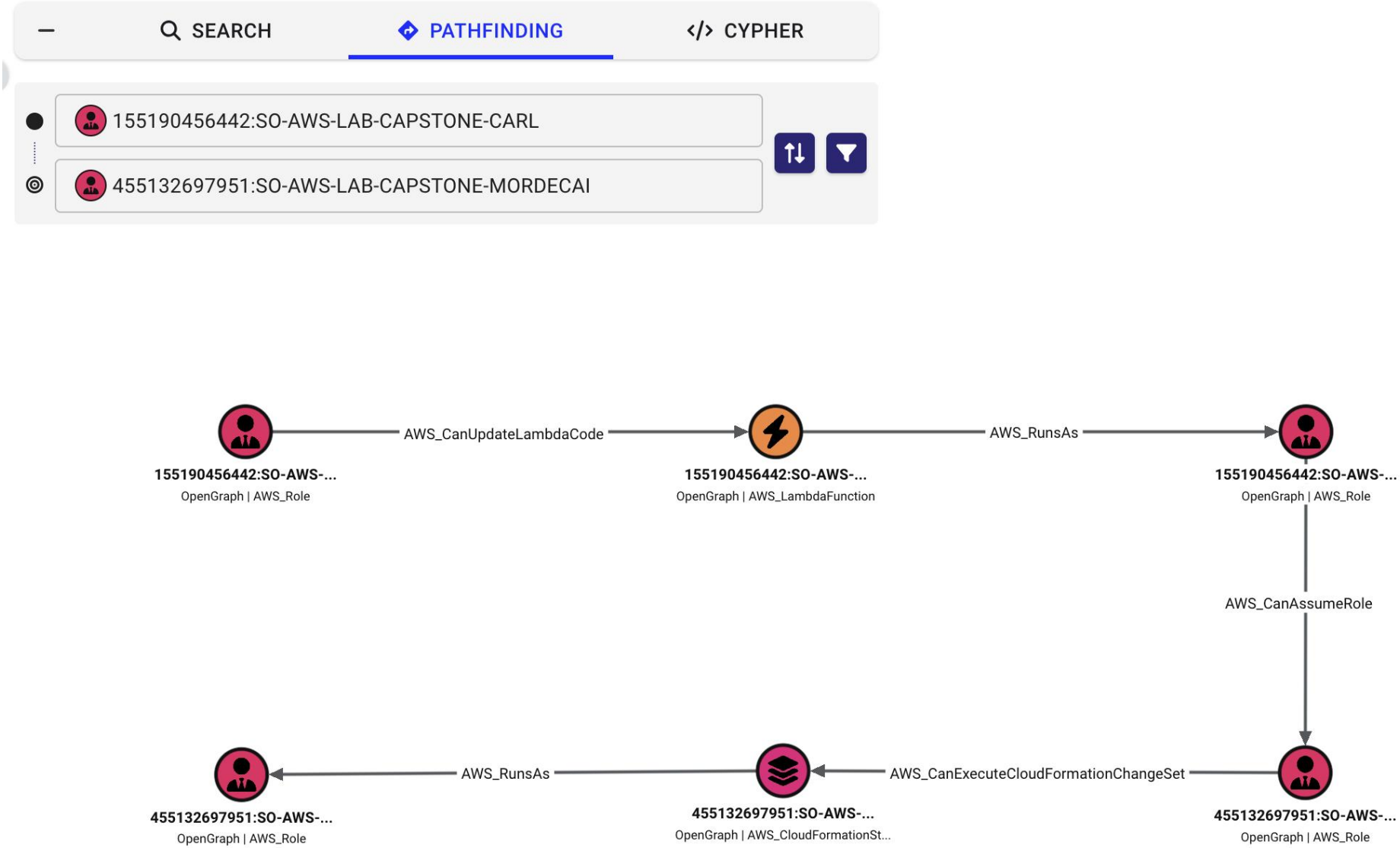
The path BloodHound draws

One curated route, hop by hop

- **`CanUpdateLambdaCode`**
  CARL overwrites a function in 155190456442
- **`RunsAs`**
  The code executes as the function's role
- **`CanAssumeRole`**
  That role pivots into 455132697951
- **`CanExecuteCloudFormationChangeSet`**
  Execute a change set on a stack
- **`RunsAs`**
  The stack acts as MORDECAI — five hops, two accounts



SPECTEROPS


SEARCH
PATHFINDING
CYPHER
155190456442:SO-AWS-LAB-CAPSTONE-CARL
455132697951:SO-AWS-LAB-CAPSTONE-MORDECAI
AWS_CanUpdateLambdaCode
AWS_RunsAs
155190456442:SO-AWS-...
OpenGraph | AWS_Role
155190456442:SO-AWS-...
OpenGraph | AWS_LambdaFunction
155190456442:SO-AWS-...
OpenGraph | AWS_Role
AWS_CanAssumeRole
AWS_RunsAs
AWS_CanExecuteCloudFormationChangeSet
455132697951:SO-AWS-...
OpenGraph | AWS_Role
455132697951:SO-AWS-...
OpenGraph | AWS_CloudFormationSt...
455132697951:SO-AWS-...
OpenGraph | AWS_Role

Capstone exercise

One hour. Ten hops. One flag.

Rules of engagement

One shared lab, three real AWS accounts

This lab is shared

- Every resource you touch carries your own student suffix.
  - Student01, Student02, etc.

- The EKS cluster and the EC2 outpost are shared

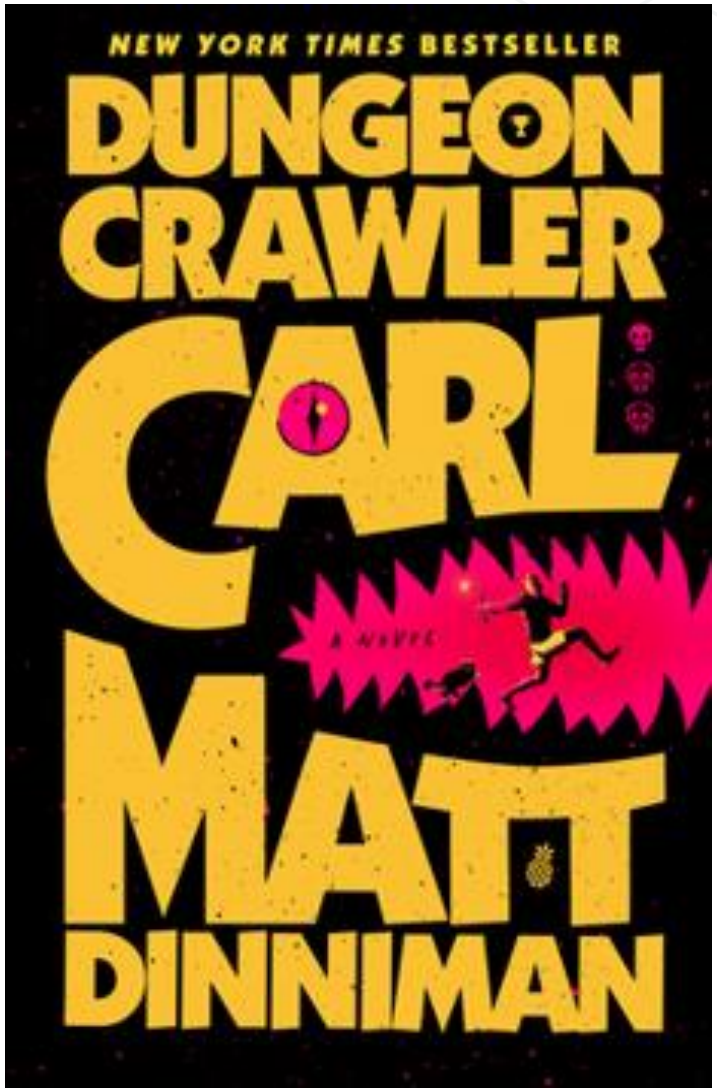
- Please be respectful of the other students!

You enter the dungeon with one role and no map

Three accounts, one locked box on the bottom floor, and no route to it on paper

Floor one: dev

- Your foothold is a role named Carl. He reads almost everything and writes almost nothing.
- The prize is incident/flag.txt in a prod evidence bucket, two accounts down.
- On paper, nobody in dev has a route to it.
- No pre-shared profiles, no benefactor, no shortcuts handed out.



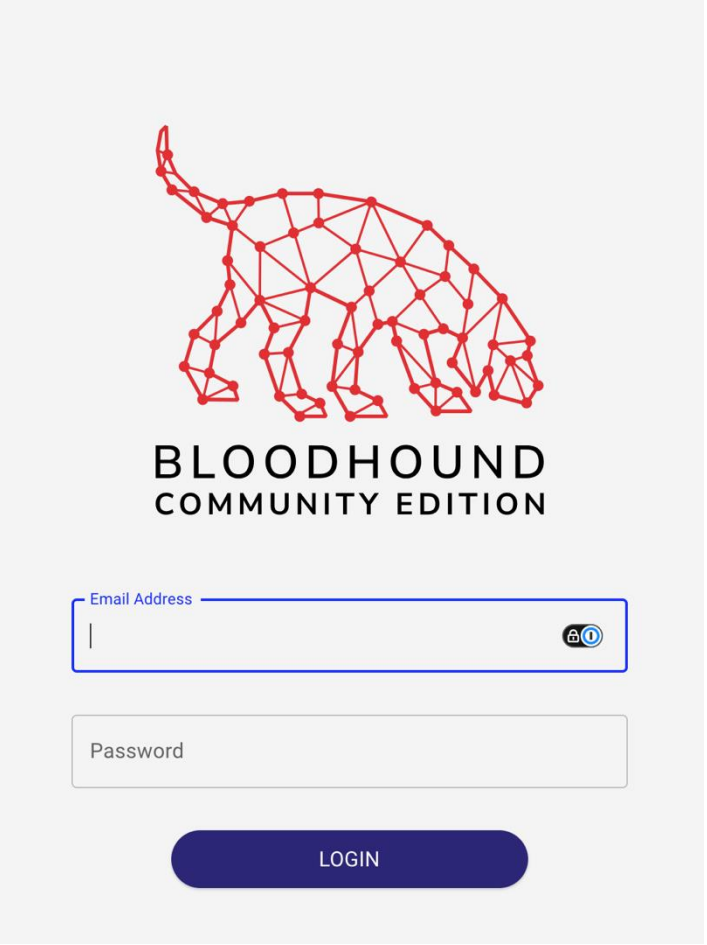
Open BloodHound and confirm the AWS graph is loaded

Access BloodHound

1. Open https://bloodhound.stormchaser.cloud/ui/explore in your browser.
2. AWSHound Schema, and graph has been preloaded into this BloodHound Community Edition instance
3. Sign in with the shared workshop account.

   `Username: demo`

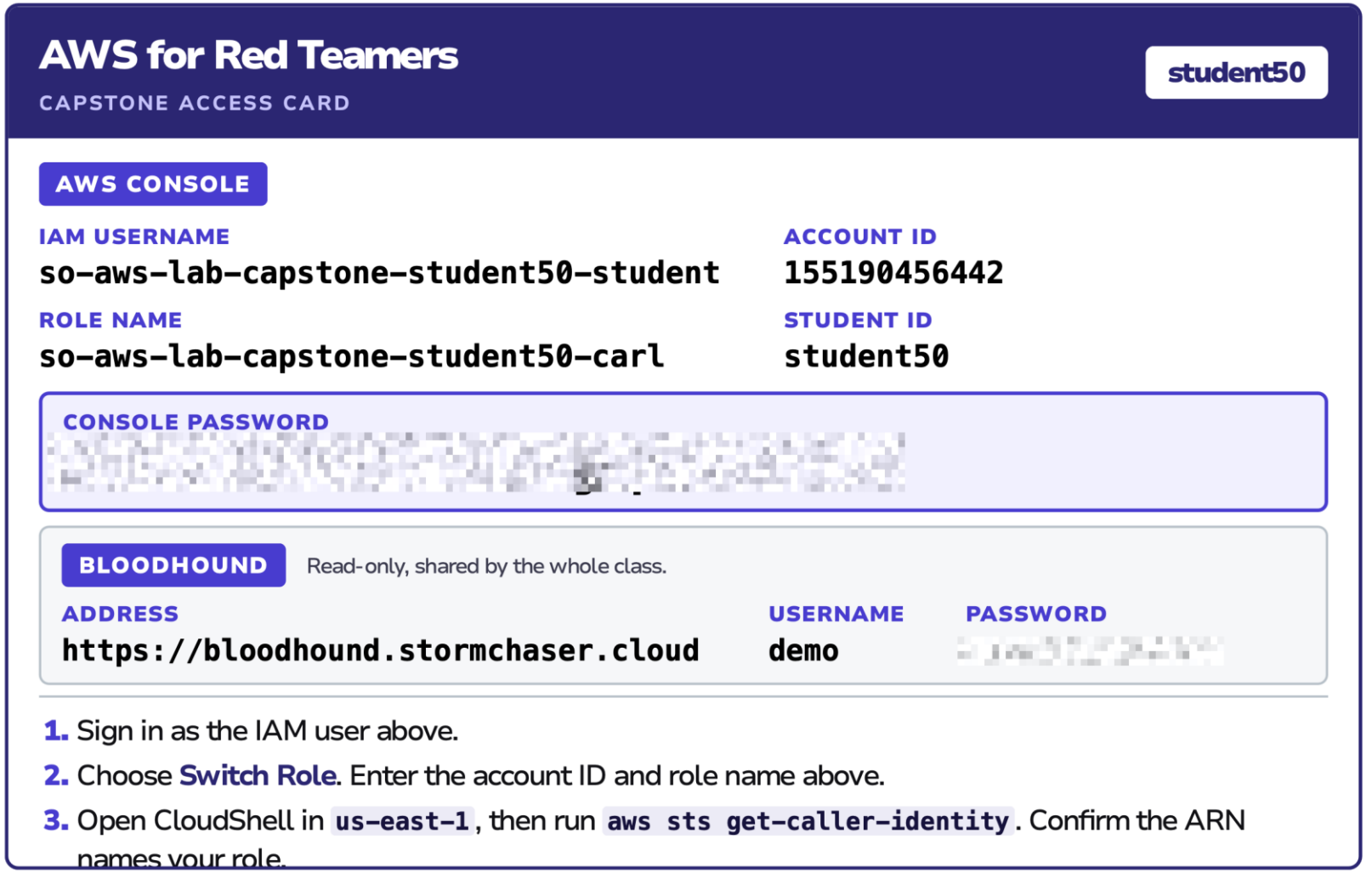
   `Password: BHUSA2026demo!`



Get into the capstone, then confirm your environment

Access the capstone lab

1. Browse to amazon and sign in with the Provided Username and Password
2. Switch roles to the entry role provided
3. Open the AWS CloudShell



Get into the capstone, then confirm your enviro

English | Contact us | AWS Marketplace | Support | My account

aws | re:Invent | Discover AWS | Products | Solutions | Pricing | Resources | Search | Sign in to console | Create account

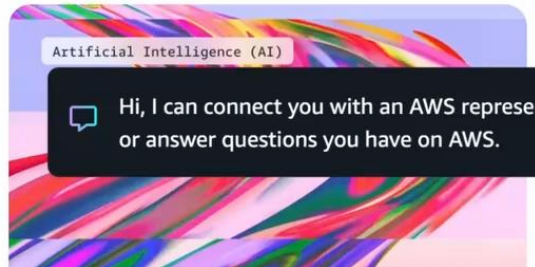
Get the greatest choice of cloud and AI capabilities

Modernize faster and scale more efficiently with an unmatched portfolio of over 240 comprehensive services

Start free with AWS

What's new

Catch up on the most talked-about launches, updates, and success stories across AWS.



Hi, I can connect you with an AWS representative or answer questions you have on AWS.

1

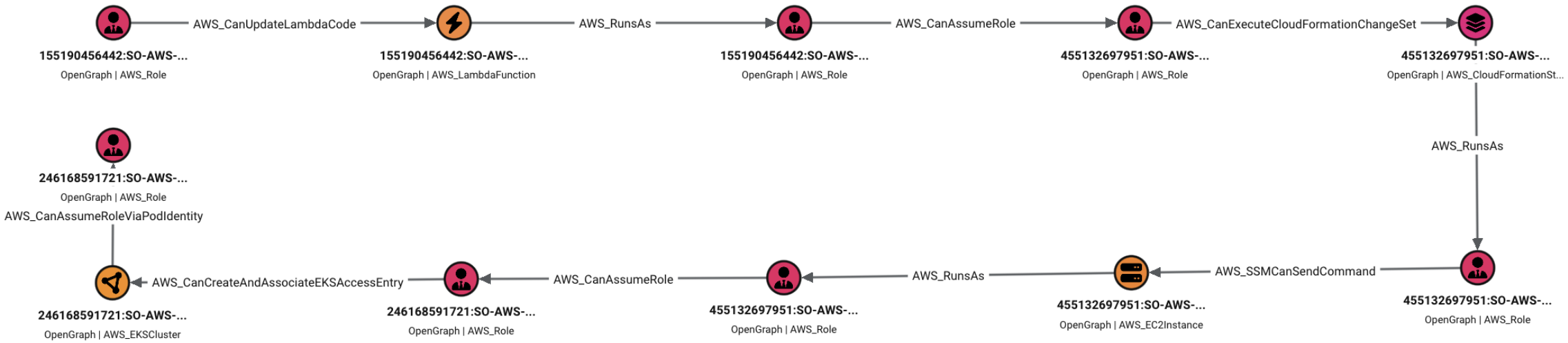
Reconstructing the Attack Path

Ten hops, three accounts

SEARCH | PATHFINDING | CYPHER

155190456442:SO-AWS-LAB-CAPSTONE-STUDENT01-CARL

246168591721:SO-AWS-LAB-CAPSTONE-STUDENT01-MONGO



Edges 1-2: Carl to Donut

The Path Begins

The Attack Path

- Carl can update the function code for a Lambda Function
- The target Lambda Function has an attached instance role
- Carl can therefore compromise the Lambda Function to obtain Donuts identity

Example:

```
aws lambda list-tags --resource <gate-golem-arn>

zip -j gate.zip index.py

aws lambda update-function-code \

  --function-name so-aws-lab-capstone-gate-golem \

  --zip-file fileb://gate.zip

aws lambda invoke --function-name ... out.json
```


Technique: lambda:UpdateFunctionCode

Rewrite a function's code and you run as its execution role

How the primitive works

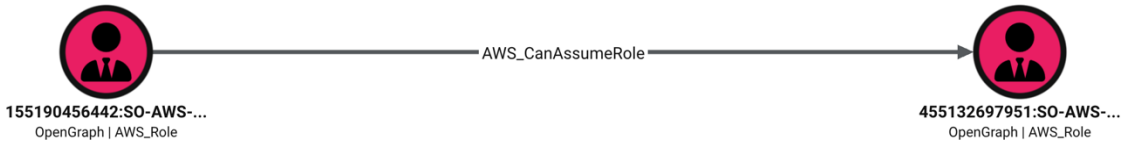
- The execution role is already attached
  - No need for the IAM:PassRole permission
- Any code you install runs with the role's credentials in its environment.
- Update the lambda function with malicious code and profit

What to flag in a real environment

- lambda:UpdateFunctionCode with Resource * or naming a privileged function.
- Execution roles carrying sts:AssumeRole, kms:Decrypt, or GetSecretValue.
- An identity that can update code and invoke the same function.
- Boundaries that block PassRole but allow lambda:UpdateFunctionCode

Edge 3: Donut to Signet

Crossing into Staging



The Attack Path

- signet's assume role trust policy document pins the donut principal.
- aws:PrincipalAccount has to equal the dev account

Example

```
aws sts assume-role --role-arn "$SIGNET_ARN" \
  --role-session-name capstone-signet
```


Technique: sts:AssumeRole

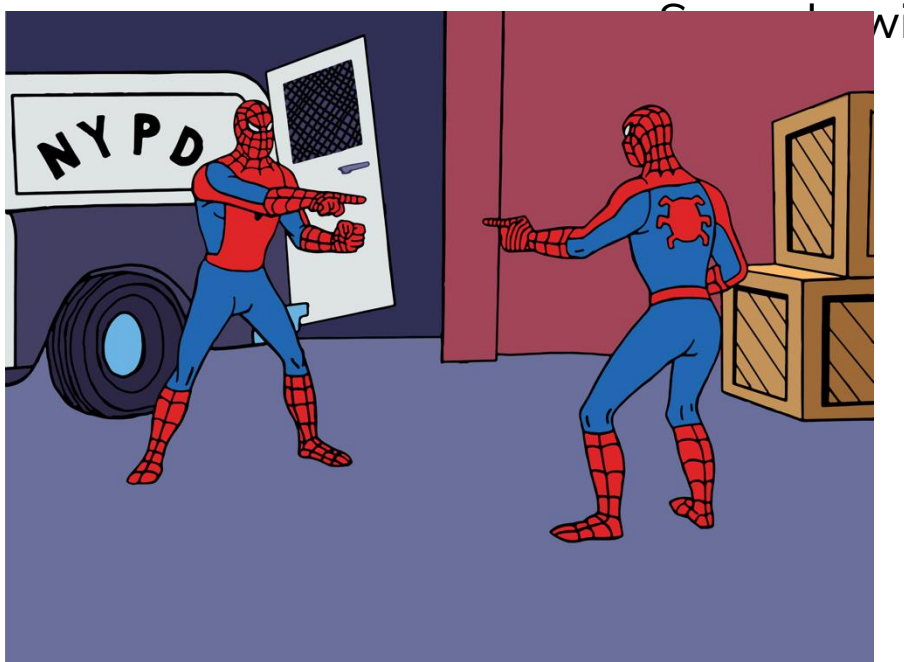
Two documents decide it: the target trust policy and your identity policy

Check the principal identity policy

- aws sts get-caller-identity, then list attached and inline policies for that user or role.
- Look for Action sts:AssumeRole with a Resource covering the target role ARN, wildcards included.
- Both sides must explicitly allow the sts:AssumeRole action

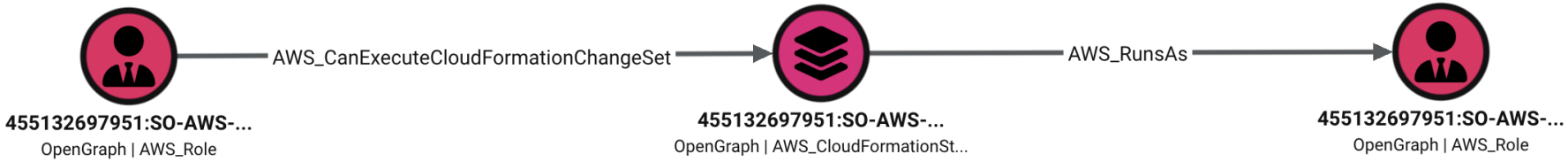
Check the target trust policy

- aws iam get-role --role-name TARGET
  - Check the AssumeRolePolicyDocument.
- Principal must name your account, user, or role ARN
- Conditions still gate it: ExternalId, MFA, or [illegible] will fail a naive call.



Edges 4-5: Signet to Mordecai

CloudFormation Abuse



The Attack Path

- The CloudFormation stack uses mordecai as its service Role.
- signet has CreateChangeSet and ExecuteChangeSet, but no iam:PassRole.
- CloudFormation, acting as mordecai, installs the handler for you.

Example: the call

```
aws cloudformation create-change-set \
  --stack-name so-aws-lab-capstone-workflow \
  --change-set-name capstone-relay \
  --change-set-type UPDATE \
  --template-body file://workflow-mutated.json
aws cloudformation execute-change-set ...
aws lambda invoke --function-name ...-relay out.json
```


Technique: cloudformation:CreateChangeSet

Change what a deployed stack does, riding the service role it already carries

How the primitive works

- CreateStack is the loud version; a change set edits a stack that exists.
- The privileged service role is already chosen and already attached.
- Propose, inspect the diff, then execute
- No iam:PassRole is required where the role is already assigned to the CloudFormation stack

What to flag in a real environment

- CreateChangeSet or ExecuteChangeSet on stacks with a privileged service role.
- Stacks whose role is broad: any change carries that role's entire policy.
- iam:PassRole without an iam:PassedToService condition pinning the service.
- Pipelines that let users execute change sets without reviewing the diff.

Edges 6-7: Mordecai to Katia

SSM Send Command

The gate

- mordecai's policy names one instance ARN and one custom document.
- AWS SSM Agent allows Mordecai to execute code on the target EC2 Instance

Example: the call

```
DOC=$(aws ssm list-documents \
  --filters Key=Owner,Values=Self ...)
aws ssm send-command --instance-ids "$OUTPOST" \
  --document-name "$DOC"
aws ssm get-parameter --name "$CREDS_PARAM" \
  --with-decryption
# the command output never contains credentials
```


Technique: ssm:SendCommand is code execution

Run a command on a managed instance and you become its instance role

How the primitive works

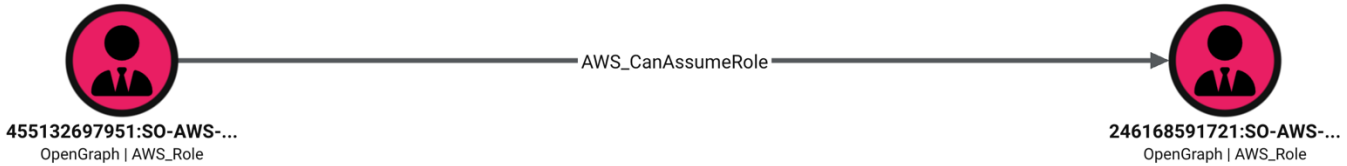
- No SSH, no key pair, no open port; the SSM agent is the direct code execution
- Commands run as root, and root reads the role's credentials from IMDS.
- Compute is identity: whoever drives the box holds the role attached to it.

What to flag in a real environment

- Any principal with ssm:SendCommand; then ask which instances it can reach.
- Instance profiles more privileged than the identities allowed to drive them.
- Instance roles granting data access or sts:AssumeRole.
- Which documents a caller may run when the shell document is denied.

Edge 8: Katia to Odette

Crossing into Production



The gate

- odette's trust pins the exact katia principal.
- aws:PrincipalAccount has to equal the staging account.
- sts:ExternalId has to match exactly, and the caller supplies it.
- The denial without it names nothing; only the trust policy explains it.

Example

```
aws sts assume-role --role-arn "$ODETTE_ARN" \
  --role-session-name capstone-odette
# AccessDenied
```

```
aws sts assume-role --role-arn "$ODETTE_ARN" \
  --role-session-name capstone-odette \
  --external-id "$EXTID"
```


Technique: sts:External Id

AWS says so; the control only works when the Principal is already tight

How the primitive works

- The external ID exists to stop confused-deputy, not to authenticate a caller.
- Being an allowed principal is not enough; the call has to carry the value.
- A broad Principal plus a readable identifier is a role you can assume.
- The value is routinely written down somewhere upstream.

What to flag in a real environment

- External IDs in SSM, S3, Terraform state, Lambda env vars, or a repo.
- Trusts relying on sts:ExternalId without also pinning the principal.
- Predictable or reused values, including ones derived from an account ID.
- The cross-account SaaS pattern generally.

Academy: 12 Conditions, Condition: sts:ExternalId

Technique: a SecureString is a lockbox with two keys

The read permission and the KMS key together are the whole primitive

How the primitive works

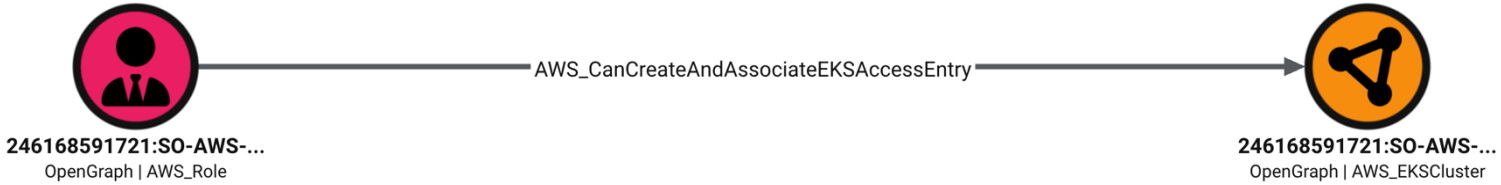
- Parameter Store is where organizations quietly park secrets and credentials.
- A SecureString is guarded by ssm:GetParameter and kms:Decrypt together.
- kms:ViaService narrows a Decrypt grant to one service path only.
- Parameter Store supplies that context; direct KMS use is not the route.

What to flag in a real environment

- Any grant of ssm:GetParameter*; then ask what lives under that path.
- SecureStrings holding JSON credentials, tokens, or connection strings.
- Roles that can decrypt the parameter's key, which is easy to grant by accident.
- ViaService-scoped grants: a foothold in that service is the actual key.

Edge 9: Odette to Production EKS Cluster

Odette to Production EKS Cluster



The gate

- eks:CreateAccessEntry on the cluster, eks:AssociateAccessPolicy on the entry.
- Register odette's role ARN, not the STS session ARN.
- The Conditions accept only AmazonEKSAdminPolicy with type=namespace.
- Cluster scope, another student's namespace, or --tags is denied.

Example

```
aws eks create-access-entry --cluster-name "$CLUSTER" \

  --principal-arn <odette-role-arn> --type STANDARD

aws eks associate-access-policy --cluster-name
"$CLUSTER" \

  --policy-arn .../AmazonEKSAdminPolicy \

  --access-scope type=namespace,namespaces=incident-
response

kubectl auth can-i '*' '*' -n incident-response
```

SPECTEROPS

Technique: an access entry is PassRole for the cluster

The entry decides which Kubernetes identity an IAM principal becomes.

How the primitive works

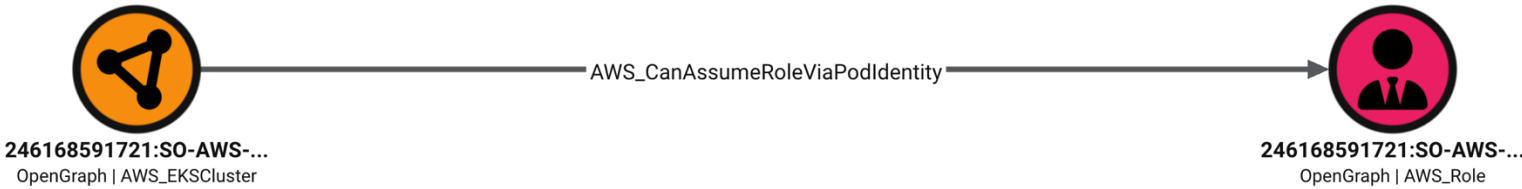
- EKS authorizes twice: IAM plus access entries, then Kubernetes RBAC.
- An access entry maps an IAM principal to a Kubernetes identity.
- Associating a cluster access policy to your own entry promotes you.
- No new credentials are involved; you are already the principal.

What to flag in a real environment

- eks:CreateAccessEntry, AssociateAccessPolicy, or UpdateAccessEntry on *.
- A principal holding an entry with zero policies; the emptiness is the setup.
- Cluster authenticationMode of API or API_AND_CONFIG_MAP.
- IAM read breadth that maps which entries a principal can modify.

Edge 10: EKS Cluster to Mongo

Assuming a Pod Identity Role



The gate

- The association already binds mongo to service account evidence-reader.
- The namespace enforces restricted Pod Security, limits, and a quota.
- Create only the service account and a compliant pod.
- The pod's projected credentials are mongo; everything after runs inside it.

Example

```
kubectl apply -f capstone-evidence-reader.yaml

kubectl wait --for=condition=Ready \

  pod/evidence-reader -n incident-response --
timeout=180s

kubectl exec -n incident-response evidence-reader -- \

  aws sts get-caller-identity
```

SPECTEROPS

Technique: pod identity is PassRole for Kubernetes

An association binds a role to one cluster, namespace, and service account

How the primitive works

- A cluster add-on injects the role's credentials into pods on that account.
- A role trusting pods.eks.amazonaws.com cannot be assumed directly.
- That makes it feel safe, which is exactly why it is worth checking.
- An association that already exists needs no create permission at all.

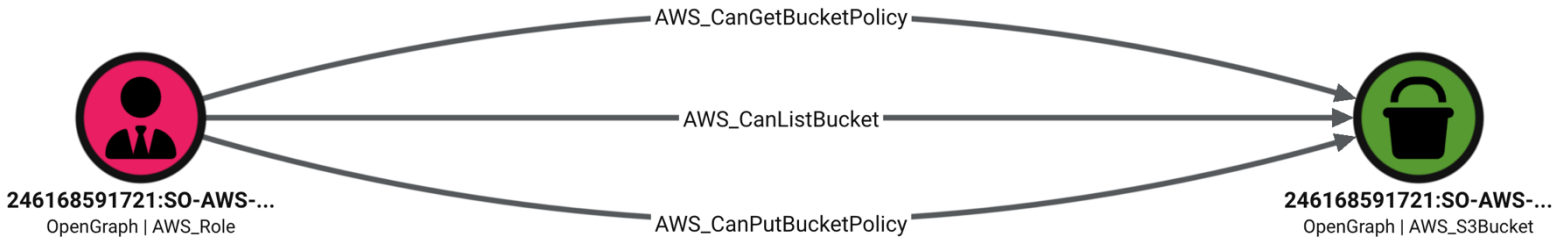
What to flag in a real environment

- eks:CreatePodIdentityAssociation paired with iam:PassRole.
- The scope of that PassRole: * means every pod-identity role in the account.
- Roles whose trust policy names pods.eks.amazonaws.com.
- Whether you can place a pod on the target service account.

Academy: 08 EKS, Abusing Pod Identity Associations

Mongo is the Final Identity

Mongo still cannot read the object



The gate

- No identity policy grants mongo s3:GetObject on the flag.
- mongo can read and replace the bucket policy, and that is the repair.
- Preserve DenyInsecureTransport and append one narrow allow.
- Exact role ARN, s3:GetObject, exact object ARN, no wildcards.

Example

```
kubectl exec ... aws s3api get-bucket-policy \
  --bucket "$BUCKET"
# append AllowMongoEvidenceObject to the saved copy
kubectl exec ... aws s3api put-bucket-policy \
  --bucket "$BUCKET" \
  --policy "$(jq -c . evidence-bucket-policy-
updated.json)"
```

SPECTEROPS

Technique: s3:PutBucketPolicy is self-authorization

You edit the lock instead of picking it

How the primitive works

- A bucket policy is resource-based and can name any principal it likes.
- Rewriting it lets you grant your own identity s3:GetObject.
- Append a statement; replacing the document drops the controls already there.
- Block Public Access does nothing against same-account principal grants.

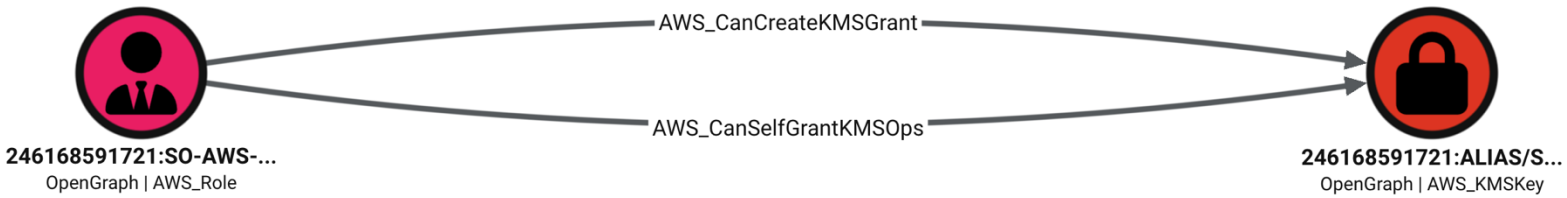
What to flag in a real environment

- Any principal with s3:PutBucketPolicy or PutBucketAcl on sensitive buckets.
- Existing policies with a broad Principal: *, account root, or wide PrincipalArn.
- PutBucketPolicy on a bucket that also holds credentials or state.
- Reliance on Block Public Access as the only control.

Academy: 10 S3, Abusing PutBucketPolicy

S3 Access isn’t Everything

S3 access is necessary and not sufficient; the key has its own policy



The gate

- Once S3 allows the read, head-object names the CMK.
- The key policy demands kms:ViaService = s3.us-east-1.amazonaws.com.
- The encryption context is the object ARN, because bucket keys are off.
- Grant Decrypt only, with EncryptionContextEquals on that exact ARN.

Example

```
CONSTRAINTS=$(jq -nc --arg o "$OBJECT_ARN" \
  '{EncryptionContextEquals:{"aws:s3:arn":$o}}')
kubectl exec ... aws kms create-grant \
  --key-id "$KMS_KEY_ARN" \
  --grantee-principal "$MONGO_ROLE_ARN" \
  --operations Decrypt --constraints "$CONSTRAINTS"
# then get-object: BORANT:<flag>
```

SPECTEROPS

Technique: kms:CreateGrant writes your own permission slip

A third authorization path, alongside key policies and IAM policies

How the primitive works

- A grant delegates specific key operations to a grantee principal.
- Hold CreateGrant and you can name yourself grantee with Decrypt.
- Grants live outside the key policy, so reviewers routinely miss them.
- Constraints can bind a grant to an exact encryption context.

What to flag in a real environment

- kms:CreateGrant on keys protecting real secrets, especially Resource *.
- Existing grants with unexpected grantees or broad Operations.
- Keys whose policy looks tight but whose CreateGrant is widely available.
- Missing encryption-context constraints on granted operations.

Academy: 11 KMS, Abusing CreateGrant

Tradecraft Academy

Where to continue after today

SPECTEROPS

Tradecraft Academy, on demand

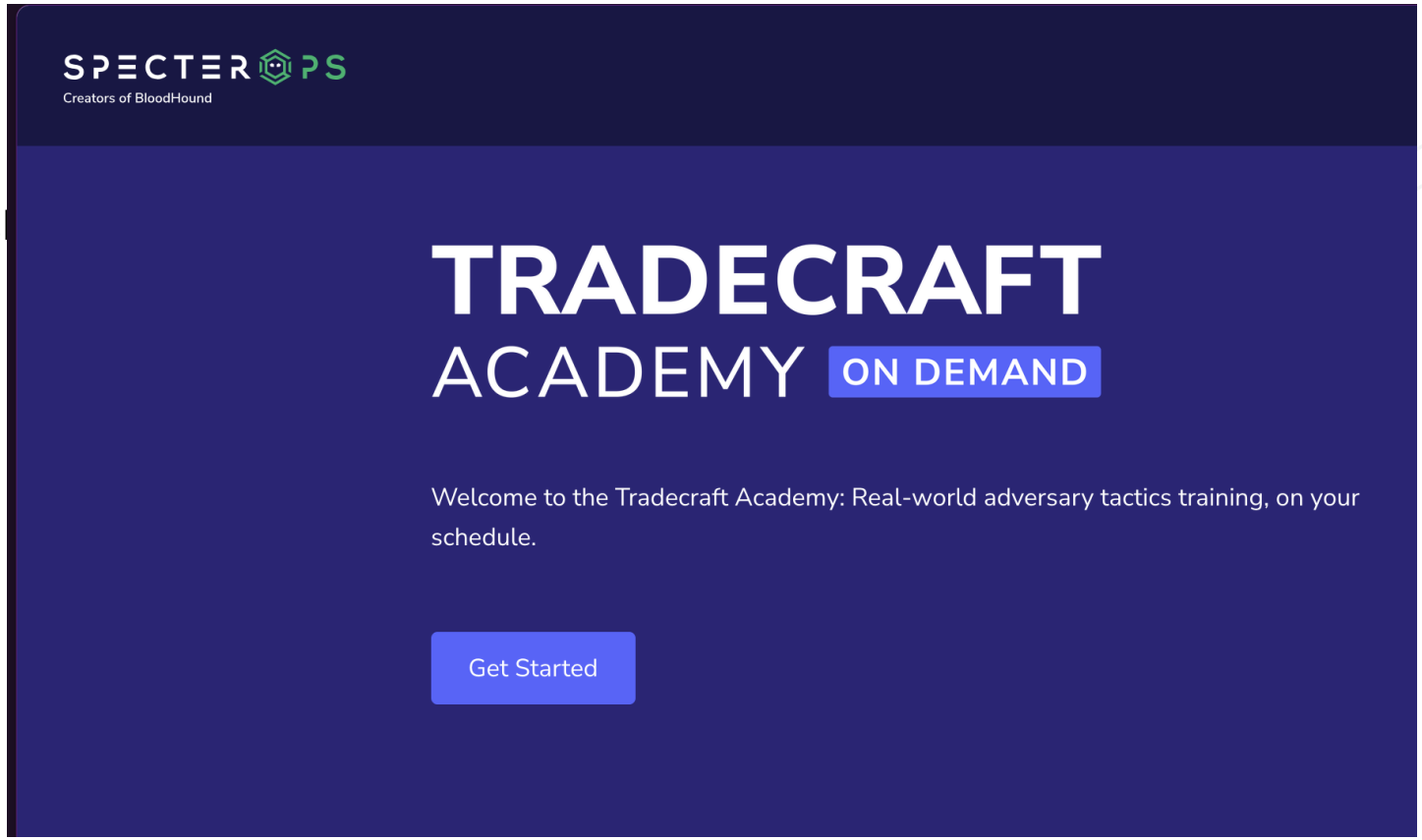
Real-world adversary tradecraft training, on your schedule

What is waiting for you

- Self-paced labs in live AWS accounts.
- The same primitives we ran today, start to finis
- New modules as the tradecraft changes.

academy.specterops.io

Click Get Started and create your account.



AWS for Red Teamers, the full course

Everything from today, plus the labs we did not have time for

What is in it

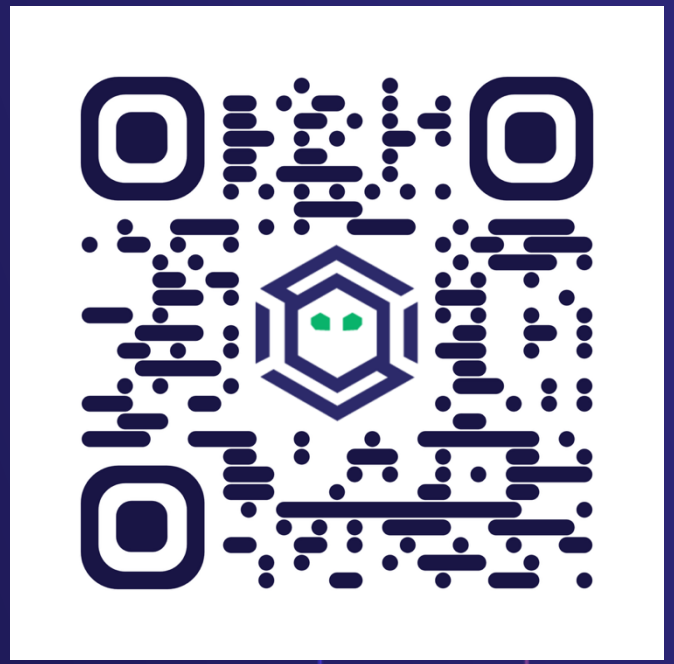
- 64 lessons across IAM, EC2, SSM, Lambda, EKS, CloudFormation, S3 and KMS.
- Hands-on labs, then the cross-account capstone.
- Today we will be starting the Capstone, but if you want to continue register for the course with this code
  - **kc-aws-2026**

Thanks for coming!

SPECTEROPS
Creators of BloodHound

Playing the Attack Path Championship?

*Use code **BH{that-was-interesting}** in-game.*
Not signed up yet? Register with code
LeadThePack



Julian Catrambone, Daniel Heinsen
SpecterOps